```
-- Q1. Create a table called employees with the following structure
--  emp_id (integer, should not be NULL and should be a primary key)
--  emp_name (text, should not be NULL)
--  age (integer, should have a check constraint to ensure the age is at least 18)
--  email (text, should be unique for each employee)
--  salary (decimal, with a default value of 30,000).
--  Write the SQL query to create the above table with all constraints

CREATE TABLE  employees (
emp_id int primary key,
emp_name text(30) NOT NULL ,
age int check(age > 18),
email varchar(30)UNIQUE,
salary decimal default (30.000)

);
select * from employees;
```

```
-- Q2.Explain the purpose of
constraints and how they help maintain
data integrity in a database. Provide
-- examples of common types of
constraints.
CREATE TABLE Employees1 (
    EmpID INT NOT NULL,
    Name VARCHAR(50) NOT NULL
);

--  Q3.Why would you apply the NOT
NULL constraint to a column? Can a
primary key contain NULL values?
Justify
-- your answer.
CREATE TABLE Employees (
    EmpID INT,
    Name VARCHAR(50) NOT NULL
);

-- Q4. Explain the steps and SQL
commands used to add or remove
constraints on an existing table.
Provide an

ALTER TABLE Employees
ADD CONSTRAINT UQ_Email UNIQUE
(Email);
```

```
-- --Q5. Explain the consequences of
attempting to insert, update, or
delete data in a way that violates
constraints.
-- Provide an example of an error
message that might occur when
violating a constraint.
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL
);

INSERT INTO Employees (EmpID, Name)
VALUES (1, NULL);   -- Error: NULL not
allowed

-- Q6. 6. You created a products table
without constraints as follows:
--  CREATE TABLE products (
--  product_id INT,
--  product_name VARCHAR(50),
--  price DECIMAL(10, 2));
--  . Now, you realise that
--  .The product_id should be a
primary key
--  .The price should have a default
value of 50.00;
```

```
CREATE TABLE products (
 product_id INT primary key,
 product_name VARCHAR(50),
price DECIMAL(10, 2) default (50.00)
);
select * from products;
--  Q7. You have two tables.
 CREATE TABLE students(
 student_id int primary key,
 student_name varchar(30),
 class_id int
);
insert into students
values (1, 'Alice' ,101),
(2, 'Bob' ,102),
(3, 'charlie' ,101);
select * from students ;
create table classes(
class_id int primary key,
class_name varchar(30)
);
insert into classes
values (101,'math') ,
(102,'science') ,
(103,'history') ;
select * from classes;
```

```
-- Write a query to fetch the
student_name and class_name for each
student using an INNER JOIN
select s.student_name, c.class_name
from students as s
INNER join classes as c
on class_id.s = class_id.c;
select * from students;
select * from classes;

--  Q8. Consider the following three
tables:
CREATE TABLE order_1
(
order_id int  primary key,
order_date date,
customer_id int
);
insert into order_1
values(1,'2024-01-01',101),
   (2,'2024-01-03',103);

create table customer_1
(
customer_id int primary key,
customer_name varchar(30)
);
insert into customer_1
```

```
values(101,'Alice'),
(102,'bob');

create table product_1(
product_id int primary key,
product_name varchar(30) ,
order_id int
);
insert into product_1
values (1,'laptop',1),
(2,'phone',null);

select * from order_1;
select * from customer_1 ;
select * from product_1;
 -- Write a query that shows all
order_id, customer_name, and
product_name, ensuring that all
products are
-- listed even if they are not
associated with an order
-- Hint: (use INNER JOIN and LEFT
JOIN)

select * from order_1;
select * from customer_1 ;
select * from product_1;
-- Query
```

```
select
o.order_id,c.customer_name,p.product_n
ame from order_1 o
INNER JOIN customer_1 c on
o.customer_id=c.customer_id
left join  product_1 p on
o.order_id=p.order_id;

-- Q9. Given the following tables:
CREATE TABLE SALES(
sales_id int primary key,
product_id int ,
amount numeric
);
insert into SALES
VALUES( 1,101,500),
(2,102,300),
(3,101,700);
SELECT * FROM SALES ;
select * from product_1;
-- Write a query to find the total
sales amount for each product using an
INNER JOIN and the SUM() function
-- Query
select
p.product_id,p.product_name,sum(s.amou
nt)as total_sales from SALES s
 INNER JOIN product_1 p
```

```
      on p.product_id = s.product_id
        group by
p.product_id,p.product_name order by
total_sales desc;

-- Q 10. You are given three tables:
SELECT * FROM customer_1;
SELECT * FROM order_1;
select * from orderdetail;
insert into orderdetail
values(1,101,2),
(3,102,1),
(2,101,3);
SELECT
    o.order_id,
    c.customer_name,
    od.quantity
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN order_details od
    ON o.order_id = od.order_id;


-- FUNCTION IN SQL
-- Q1. Identify the primary keys and
foreign keys in maven movies db.
Discuss the differences
```

;
actor → actor_id
film → film_id
customer → customer_id
rental → rental_id
payment → payment_id
store → store_id
inventory → inventory_id
category → category_id
staff → staff_id
address → address_id
city → city_id
country → country_id

-- Q2.List all details of actors
actor_id → unique ID of the actor (Primary Key)
first_name → actor's first name
last_name → actor's last name
last_update → timestamp of the last modification

-- Q3.List all customer information from DB

customer_id → unique ID of each customer (PK)

store_id → references which store the customer belongs to (FK)
first_name
last_name
email
address_id → links to the address table (FK)
active → status (active/inactive)
create_date
last_update

Q4 List different countries.

country_id → unique ID for each country (PK)
country → name of the country
last_update → timestamp of the last modification

```
-- Q5.Display all active customers.
SELECT *
FROM customer
WHERE active = 1;
-- Q6-List of all rental IDs for customer with ID 1
SELECT rental_id
FROM rental
```

```
WHERE customer_id = 1;
-- Q7 Display all the films whose
rental duration is greater than 5

SELECT film_id, title, rental_duration
FROM film
WHERE rental_duration > 5;

-- Q8 List the total number of films
whose replacement cost is greater than
$15 and less than $20

SELECT COUNT(*) AS total_films
FROM film
WHERE replacement_cost > 15
  AND replacement_cost < 20;


-- Q9- Display the count of unique
first names of actors
SELECT COUNT(DISTINCT first_name) AS
unique_first_names
FROM actor;

-- Q10- Display the first 10 records
from the customer table
SELECT *
FROM customer
```

```
LIMIT 10;

-- 11.Display the first 3 records from the customer table whose first name starts with ‘b’.
SELECT *
FROM customer
WHERE first_name LIKE 'B%'
LIMIT 3;


-- 12.Display the names of the first 5 movies which are rated as ‘G’.

SELECT title
FROM film
WHERE rating = 'G'
LIMIT 5;

-- 13. Find all customers whose first name starts with "a".

SELECT *
FROM customer
WHERE first_name LIKE 'A%';

-- 14. Find all customers whose first name ends with "a".
```

```
SELECT *
FROM customer
WHERE first_name LIKE '%a';

-- 15  Display the list of first 4
cities which start and end with ‘a’ .

SELECT city
FROM city
WHERE city LIKE 'A%a'
LIMIT 4;

-- 16 Find all customers whose first
name have "NI" in any position.

SELECT *
FROM customer
WHERE first_name LIKE '%NI%';

-- 17 Find all customers whose first
name have "r" in the second position .

SELECT *
FROM customer
WHERE first_name LIKE '_r%';
```

```
-- 18.Find all customers whose first
name starts with "a" and are at least
5 characters in length.

SELECT *
FROM customer
WHERE first_name LIKE 'A%'
  AND LENGTH(first_name) >= 5;

-- 19 Find all customers whose first
name starts with "a" and ends with
"o".

SELECT *
FROM customer
WHERE first_name LIKE 'A%o';

-- 20  Get the films with pg and pg-13
rating using IN operator.
SELECT *
FROM film
WHERE rating IN ('PG', 'PG-13');

-- 21  Get the films with length
between 50 to 100 using between
operator.
```

```
SELECT *
FROM film
WHERE length BETWEEN 50 AND 100;


-- 22 Get the top 50 actors using
limit operator.

SELECT *
FROM actor
LIMIT 50;


--  23 - Get the distinct film ids
from inventory table.

SELECT DISTINCT film_id
FROM inventory;

-- Functions
--  Basic Aggregate Functions:
--  Question 1:
--  Retrieve the total number of
rentals made in the Sakila database.
--  Hint: Use the COUNT()
function.alter

SELECT COUNT(*) AS total_rentals
```

```
FROM rental;

--  Question 2:
--  Find the average rental duration
(in days) of movies rented from the
Sakila database.
--  Hint: Utilize the AVG() function.

SELECT AVG(rental_duration) AS
avg_rental_duration
FROM film;

-- String Functions:
--  Question 3:
--  Display the first name and last
name of customers in uppercase.
--  Hint: Use the UPPER () function.

SELECT UPPER(first_name) AS
first_name_upper,
       UPPER(last_name) AS
last_name_upper
FROM customer;


-- Question 4:
```

```
--  Extract the month from the rental
date and display it alongside the
rental ID.
--  Hint: Employ the MONTH() function.
SELECT rental_id,
       MONTH(rental_date) AS
rental_month
FROM rental;

-- GROUP BY:
--  Question 5:
--  Retrieve the count of rentals for
each customer (display customer ID and
the count of rentals).
--  Hint: Use COUNT () in conjunction
with GROUP BY.

SELECT customer_id,
       COUNT(rental_id) AS
total_rentals
FROM rental
GROUP BY customer_id;


--  Question 6:
--  Find the total revenue generated
by each store.
--  Hint: Combine SUM() and GROUP BY
```

```
SELECT s.store_id,
       SUM(p.amount) AS total_revenue
FROM payment p
JOIN staff s ON p.staff_id =
s.staff_id
GROUP BY s.store_id;

--  Question 7:
--  Determine the total number of
rentals for each category of movies.
--  Hint: JOIN film_category, film,
and rental tables, then use cOUNT ()
and GROUP BY

SELECT c.name AS category_name,
       COUNT(r.rental_id) AS
total_rentals
FROM rental r
JOIN inventory i ON r.inventory_id =
i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id =
fc.film_id
JOIN category c ON fc.category_id =
c.category_id
GROUP BY c.name
ORDER BY total_rentals DESC;
```

```
--  Question 8:
--  Find the average rental rate of
movies in each language.
--  Hint: JOIN film and language
tables, then use AVG () and GROUP BY.

SELECT l.name AS language_name,
       AVG(f.rental_rate) AS
avg_rental_rate
FROM film f
JOIN language l ON f.language_id =
l.language_id
GROUP BY l.name
ORDER BY avg_rental_rate DESC;


-- Joins


--  Questions 9 -
--  Display the title of the movie,
customer s first name, and last name
who rented it.
--  Hint: Use JOIN between the film,
inventory, rental, and customer tables

select * from film;
select * from customer;
```

```
select * from inventory;
select * from rental;
select f.title as movis_title,
c.first_name as first_name,
c.last_name as last_name
from film as f
join inventory i on
i.film_id=f.film_id
join rental r on
i.inventory_id=r.inventory_id
join customer c on
r.customer_id=c.customer_id;

--  Question 10:
--  Retrieve the names of all actors
who have appeared in the film "Gone
with the Wind."
--  Hint: Use JOIN between the film
actor, film, and actor tables.
select * from actor;
select * from film;
select * from film_actor ;

select a.first_name as first_name,
a.last_name as last_name
 from film_actor fa
join actor a on fa.actor_id=a.actor_id
join film f on fa.film_id=f.film_id
```

```
where f.title='gone with the wind';

--  Question 11:
--  Retrieve the customer names along
with the total amount they've spent on
rentals.
--  Hint: JOIN customer, payment, and
rental tables, then use SUM() and
GROUP BY
select * from payment;
select * from rental;
select * from customer;
select c.first_name as
customer_first_name,
       c.last_name as
customer_last_name,
       sum(p.amount) as total_spend
        from payment p
join customer c on
p.customer_id=c.customer_id
join rental r on
p.rental_id=r.rental_id
group by
c.customer_id,c.first_name,c.last_name
order by total_spend desc;

--  Question 12:
```

```
--  List the titles of movies rented
by each customer in a particular city
(e.g., 'London').
--  Hint: JOIN customer, address,
city, rental, inventory, and film
tables, then use GROUP BY
select * from film ;
select * from customer;
select * from address;
select * from city;
select * from rental ;
select * from inventory;

select c.first_name as
customer_first_name ,
c.last_name as customer_last_name ,
f.title as movies
from customer c
join address a on
c.address_id=a.address_id
join city ci on a.city_id=ci.city_id
join rental r on
c.customer_id=r.customer_id
join inventory i on
r.inventory_id=i.inventory_id
join film f on i.film_id=f.film_id
where ci.city= 'London'
```

```
group by
c.first_name,c.last_name,f.title
 order by
c.first_name,c.last_name,f.title;

-- Advanced Joins and GROUP BY

-- . Q1. Rank the customers based on
the total amount they've spent on
rentals.

SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ',
c.last_name) AS customer_name,
    SUM(p.amount) AS total_spent,
    RANK() OVER (ORDER BY
SUM(p.amount) DESC) AS rank_position
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name,
c.last_name
ORDER BY total_spent DESC;

-- Q2.2. Calculate the cumulative
revenue generated by each film over
time.
```

```
SELECT
    f.film_id,
    f.title,
    DATE(p.payment_date) AS
revenue_date,
    SUM(p.amount) AS daily_revenue,
    SUM(SUM(p.amount)) OVER (
        PARTITION BY f.film_id
        ORDER BY DATE(p.payment_date)
    ) AS cumulative_revenue
FROM film f
JOIN inventory i
    ON f.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
JOIN payment p
    ON r.rental_id = p.rental_id
GROUP BY f.film_id, f.title,
DATE(p.payment_date)
ORDER BY f.film_id, revenue_date;


-- Q3  Determine the average rental
duration for each film, considering
films with similar lengths.

SELECT
```

```
    f.film_id,
    f.title,
    f.length AS film_length,
    AVG(DATEDIFF(r.return_date,
r.rental_date)) AS
avg_rental_duration,
    AVG(AVG(DATEDIFF(r.return_date,
r.rental_date))) OVER (
        PARTITION BY f.length
    ) AS
avg_duration_for_similar_length
FROM film f
JOIN inventory i
    ON f.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.film_id, f.title, f.length
ORDER BY f.length, f.title;

-- Q4. Identify the top 3 films in
each category based on their rental
counts.

SELECT
    c.name AS category_name,
    f.film_id,
    f.title,
```

```
    COUNT(r.rental_id) AS
rental_count,
    RANK() OVER (
        PARTITION BY c.category_id
        ORDER BY COUNT(r.rental_id)
DESC
    ) AS film_rank
FROM category c
JOIN film_category fc
```

STUDENT NAME :MD IZHAR AHMAD
DOMAIN : DATA ANALYST
ASSIGNMENT NAME: SQL DATABASE
DATE:27/09/2025